MACHINE LEARNING

Interactive Algorithms, Visual Playgrounds & Practical Links

Purpose: Learn algorithms by experimenting with data, parameters, graphs and decision boundaries—not just memorizing definitions.

HOW TO USE THIS GUIDE

1. Open a playground link. 2. Select an algorithm or lesson. 3. Change data points and settings. 4. Observe how the model or graph changes. 5. Recreate the same experiment in Python with scikit-learn.

Coverage note: Not every website simulates every algorithm interactively. Some provide hands-on playgrounds, while others offer visual explanations or guided labs. Check the resource itself for current coverage.

1. Interactive ML playgrounds

Resource	What to explore	Link
ML Playground	Interactive lessons and demos for classical ML concepts and algorithms.	Open website
TensorFlow Playground	Experiment with neural-network layers, neurons, activation functions, learning rate and decision boundaries.	Open website
ML Hive	Interactive machine-learning playgrounds and visual learning resources.	Open website
ML / AI Arcade	Algorithm visualizations and experiments, particularly useful for clustering and classification concepts.	Open website
MLU-Explain (Amazon)	Visual explanations of ML concepts such as logistic regression, decision trees and model evaluation.	Open website
ExplAIn	Interactive AI/ML lessons and labs for core concepts and model behavior.	Open website
Google Teachable Machine	Train image, audio or pose classification models in a no-code interface.	Open website
Seeing Theory	Interactive probability and statistics visualizations—useful mathematical foundations for ML.	Open website
Distill	Visual and interactive explanations of machine learning and neural-network ideas.	Open website

2. Algorithms: what to visualize and where to start

Regression

Linear Regression: Fit a straight line; change points and observe slope, intercept and prediction error. [ML Playground; scikit-learn]

Polynomial Regression: Add polynomial features and see how a curved fit changes; watch for overfitting. [ML Playground; scikit-learn]

Ridge / Lasso / Elastic Net: Compare regularization and how coefficients shrink. [scikit-learn examples; visual lessons]

Decision Tree Regressor: Watch the feature space split into regions and predict values. [scikit-learn; MLU-Explain concepts]

Random Forest Regressor: Compare many trees and their averaged predictions. [scikit-learn; guided labs]

Gradient Boosting / AdaBoost: Observe sequential learners correcting earlier errors. [scikit-learn; guided labs]

Support Vector Regression (SVR): Explore margin/epsilon tube intuition and support vectors. [scikit-learn; visual lessons]

KNN Regression: Change K and see how nearby points influence a prediction. [ML Playground where available; scikit-learn]

Classification

Logistic Regression: Move labeled points and inspect the decision boundary and probability outputs. [ML Playground; MLU-Explain]

K-Nearest Neighbors (KNN): Change K and distance behavior; see how the classification region changes. [ML Playground where available]

Support Vector Machine (SVM / SVC): Explore margins, support vectors and linear vs. nonlinear boundaries. [ML Playground where available]

Decision Tree Classifier: Follow feature-based splits and see how depth changes the boundary. [MLU-Explain; scikit-learn]

Random Forest Classifier: Compare the ensemble result with an individual decision tree. [scikit-learn; guided labs]

Naive Bayes (Gaussian / Multinomial / Bernoulli): Understand class probabilities and feature likelihoods. [Visual lessons; scikit-learn]

Gradient Boosting / AdaBoost / Extra Trees: Explore ensemble voting or sequential correction. [scikit-learn; guided labs]

MLP Classifier: Inspect a neural network's learned classification boundary. [TensorFlow Playground for neural-network intuition]

Clustering and dimensionality reduction

K-Means: Move points, choose K and watch cluster centers update. [ML Playground / ML Arcade where available]

DBSCAN: Change neighborhood radius and minimum samples to see density-based clusters and noise. [ML Arcade where available]

Agglomerative / Hierarchical Clustering: Explore how clusters merge into a hierarchy or dendrogram. [Visual lessons; scikit-learn]

Mean Shift / OPTICS / BIRCH: Compare density or hierarchical clustering behavior. [scikit-learn; guided labs]

PCA: Reduce dimensions and inspect how much variance principal components retain. [ML Arcade where available; scikit-learn]

t-SNE: See how high-dimensional samples can be arranged for visualization. [Visual lessons; scikit-learn]

Truncated SVD / NMF / ICA: Explore alternative matrix factorization and component representations. [scikit-learn; visual lessons]

3. Core ML workflow: functions to practice in Python

Data handling: pandas.read_csv(), DataFrame.head(), DataFrame.info(), DataFrame.describe(), dropna(), fillna().

Preprocessing: StandardScaler(), MinMaxScaler(), RobustScaler(), OneHotEncoder(), OrdinalEncoder(), LabelEncoder(), SimpleImputer(), PolynomialFeatures().

Splitting and tuning: train_test_split(), cross_val_score(), KFold(), StratifiedKFold(), GridSearchCV(), RandomizedSearchCV().

Model API: model.fit(X_train, y_train), model.predict(X_test), model.score(X_test, y_test).

Regression metrics: mean_absolute_error(), mean_squared_error(), root_mean_squared_error(), r2_score().

Classification metrics: accuracy_score(), precision_score(), recall_score(), f1_score(), confusion_matrix(), classification_report(), roc_auc_score().

4. Mathematical ideas worth seeing visually

Linear model: $y = mx + c$. Sigmoid: $\sigma(z) = 1 / (1 + e^{-z})$. ReLU: $\max(0, z)$.

Loss functions: Mean Squared Error (MSE), Mean Absolute Error (MAE), cross-entropy loss. Optimization: gradient descent and learning rate.

Other foundations: vectors, matrices, dot product, matrix multiplication, transpose, eigenvectors, SVD, mean, variance, standard deviation, covariance and correlation.

5. Suggested hands-on learning sequence

Order	Topic	Practical task
Step 1	Linear Regression	Fit a line, change points, and understand residuals and MSE.
Step 2	Logistic Regression	Change class labels and inspect probabilities and decision boundaries.
Step 3	KNN and SVM	Experiment with K, distance, margins and kernels.
Step 4	Decision Tree and Random Forest	Change tree depth and compare one tree with an ensemble.
Step 5	K-Means and DBSCAN	Change cluster count, radius and density.
Step 6	PCA and t-SNE	Explore dimensionality reduction and 2D projections.
Step 7	Neural Networks	Use TensorFlow Playground to change layers, neurons and learning rate.
Step 8	Python implementation	Rebuild each experiment using scikit-learn and evaluate it.

6. Official Python documentation and datasets

Scikit-learn user guide: Open documentation

Scikit-learn API reference: Browse estimators and functions

NumPy documentation: Open NumPy

Pandas documentation: Open Pandas

Matplotlib tutorials: Open tutorials

Seaborn examples: Open examples

UCI Machine Learning Repository: Find practice datasets

Kaggle datasets: Find practice datasets

Final learning rule: For every algorithm, do three things: (1) visualize it, (2) write a small Python implementation or use its scikit-learn estimator, and (3) explain in your own words what changes when you alter its parameters.